

Guía de Estudio -- Módulo I: Diseño Ágil + Python

Tema 01 . Laboratorio de Programación y Lenguajes 2026

Universidad Nacional de Tierra del Fuego -- Instituto IDEI

Alumnos del IF009 -- Semanas 2-3

Docente

Prof. Matias Gel

2026

Objetivos de esta guía

Al finalizar este módulo, puedes:

#	Objetivo	Cómo se evalúa
OA1	Describir el modelo ágil y sus herramientas (VS Code, Git, CI)	Flujo de trabajo con commits y CI en verde
OA2	Escribir funciones Python 3.13 con tipos, condicionales y loops	Scripts [src/] con pruebas automatizadas en verde
OA3	Usar colecciones Python para resolver problemas concretos	Soluciones con colecciones claras y eficientes
OA4	Organizar código en módulos con docstrings, Ruff-compliant y type hints	Estructura [src/tests/] y documentación de funciones
OA5	Usar Copilot con prompts estructurados y documentarlos en PROMPTS.md	[PROMPTS.md] con ≥ 3 prompts completos
OA6	Explicar que hace el código generado por IA antes de entregarlo	Calidad del PROMPTS.md

Conceptos previos necesarios

- [x] Cuenta GitHub activa con SSH configurado (Tema 00)
 - [x] GitHub Codespaces funcional (Tema 00)
 - [x] Git básico: [add], [commit], [push] (Tema 00)
 - [x] Bootstrap 5 + HTML básico (Tema 00 -- solo de contexto)
-

PARTE 1: El Modelo Ágil

1.1 ¿Que es el desarrollo ágil?

El desarrollo ágil es una forma de construir software que prioriza iteraciones cortas, feedback continuo y colaboración directa sobre planificación exhaustiva y documentación extensa.

Surge en 2001 cuando 17 desarrolladores publicaron el Manifiesto Ágil (agilemanifesto.org), que resume su filosofía en 4 valores:

Valoramos más	Que
Individuos e interacciones	Procesos y herramientas
Software funcionando	Documentación exhaustiva
Colaboración con el cliente	Negociación de contratos
Respuesta a los cambios	Seguir un plan rígido

Importante: Ágil no significa "sin planificación". Significa que la planificación es iterativa y se ajusta continuamente.

1.2 Cascada vs. Ágil

El modelo cascada (waterfall) fue el dominante durante los 1970s-1990s. Propone fases secuenciales: Requisitos -> Diseño -> Implementación -> Prueba -> Despliegue. El problema: el cliente ve el software por primera vez cuando ya está "terminado" y difícil de cambiar.

En este curso: Tienes tests preescritos (los "requisitos") y tu tarea es hacer que todos pasen haciendo iteraciones pequeñas (commits). Eso es el ciclo ágil más mínimo posible: Red -> Green -> Refactor.

1.3 El entorno de desarrollo ágil

Para este curso usamos GitHub Codespaces como entorno estándar. Esto significa:

- No necesitas instalar Python localmente -- todo corre en un servidor de GitHub
- El entorno está definido en [.devcontainer/devcontainer.json] -- es code-as-infrastructure
- Cualquier alumno (y el CI) abre exactamente el mismo entorno

> Extensiones VS Code instaladas automáticamente

Extensión	Para que sirve
Pylance	Type checking en tiempo real -- muestra errores de tipo sin ejecutar
Ruff	Linter automático -- marca violaciones PEP 8 al guardar
GitLens	Muestra historial Git en el margen del editor
GitHub Copilot	Asistente de código con IA (usar con PROMPTS.md)

2.1 Por que Python 3.13

Python 3.13 (octubre 2024) trae mejoras importantes para aprender:

- REPL mejorado: historial persistente, colores, mejor soporte para código multilínea
- Errores contextuales: [Did you mean: 'upper'?] cuando escribís [upper_case()]
- Mensajes de error más claros con contexto sobre dónde ocurrió el error

Para abrir el REPL desde tu Codespace:

```
python3
```

bash

2.2 Tipos de datos primitivos

Python es un lenguaje de tipado dinámico: no declarás el tipo explícitamente, Python lo infiere. Sin embargo, podes (y en este curso debes) agregar type hints para documentar los tipos.

> Los 5 tipos primitivos

Tipo	Ejemplos	Características
[int]	[42], [-7], [1000000]	Precisión arbitraria (sin overflow)
[float]	[3.14], [-0.5], [1e10]	IEEE 754 -- puede tener errores de redondeo
[str]	["hola"], ['mundo']	Inmutable -- no se puede modificar in-place
[bool]	[True], [False]	Subclase de [int] (True=1, False=0)
[None]	[None]	Ausencia de valor -- tipo [NoneType]

```
# Ejemplos de declaración con type hints
nombre: str = "Ada Lovelace"
edad: int = 25
promedio: float = 7.85
activo: bool = True
email: str | None = None # Puede ser string o None
```

python

2.3 Operadores aritmeticos

```
# Operadores estándar
10 + 3    # 13
10 - 3    # 7
10 * 3    # 30
10 / 3    # 3.3333... (división siempre devuelve float)

# Operadores especiales de Python
10 // 3    # 3    -> división entera (floor division)
10 % 3     # 1    -> módulo (resto de la división)
2 ** 8     # 256  -> potencia
```

Trampa común: `10 / 2` devuelve `5.0` (float), no `5` (int). Si necesitas un entero: `10 // 2`.

2.4 Strings -- inmutabilidad e interpolación

Los strings en Python son inmutables: no puedes cambiar un carácter individual. Para "modificarlos", creas un string nuevo.

```
# [X] Intentar modificar un string
saludo = "hola"
saludo[0] = "H"    # TypeError: 'str' does not support item assignment

# [OK] Crear un nuevo string
saludo_formal = saludo.capitalize()    # "Hola" (nuevo string)
print(saludo)                          # "hola" (sin cambios)
```

> f-strings -- la forma de interpolar

```
nombre = "Ada"
edad = 25
texto = f"Hola, {nombre}. Tienes {edad} años."
# "Hola, Ada. Tienes 25 años."

# Expresiones dentro del f-string
pi = 3.14159
print(f"Pi redondeado: {pi:.2f}")    # "Pi redondeado: 3.14"
```

> Metodos de string más usados

```
texto = "  Hola Mundo  "
texto.strip()          # "Hola Mundo" -- quita espacios extremos
texto.lower()          # " hola mundo "
texto.upper()          # "  HOLA MUNDO  "
"a,b,c".split(",")     # ["a", "b", "c"]
",".join(["a", "b", "c"]) # "a,b,c"
"Python".startswith("Py") # True
"Python".endswith("on")  # True
"Python".replace("y", "Y") # "PYthon"
```

2.5 Variables y referencias

Concepto crucial: En Python, las variables son etiquetas que apuntan a objetos en memoria -- no son "cajas" que contienen valores.

```
# El problema de la referencia compartida
a = [1, 2, 3]
b = a          # b apunta al MISMO objeto que a

b.append(99)
print(a)       # [1, 2, 3, 99] <- a tambien cambi6

# Soluci6n: copiar expl6citamente
c = a.copy()   # crea un nuevo objeto
c.append(100)
print(a)       # [1, 2, 3, 99] <- a no cambi6
```

En este curso: Si tienes una funci6n que recibe una lista, ten cuidado de no modificarla si el test espera que el original quede intacto.

2.6 None y Boolean

```
# None -- ausencia de valor
resultado = None
if resultado is None:    # [OK] usar "is", no "=="
    print("Sin resultado a6n")

# Boolean -- sorpresas
True + True             # 2 (bool es subclase de int)

# Valores "falsy" (eval6an como False en un if)
bool("")                # False -- string vac6o
bool([])                # False -- lista vac6a
bool(0)                 # False
bool(None)              # False
# Todo lo dem6s es truthy

# Por eso esto funciona:
lista = []
if not lista:            # equivale a: if len(lista) == 0
    print("Lista vac6a")
```

PARTE 3: Control de Flujo

3.1 Condicionales -- if / elif / else

```
def clasificar_nota(nota: int) -> str:
    """Clasifica una nota numerica en una categor6a textual."""
    if nota >= 90:
        return "Sobresaliente"
    elif nota >= 70:
        return "Aprobado"
    elif nota >= 50:
        return "Regular"
    else:
        return "Desaprobado"
```

Reglas de estilo (PEP 8):

- 4 espacios de indentaci6n (nunca tabs)
- No usar parentesis innecesarios en la condici6n: [if x > 0:] no [if (x > 0):]

- Para valores booleanos: [if activo:] no [if activo == True:]

3.2 Bucles -- for

```
# Sobre una lista
frutas = ["manzana", "banana", "pera"]
for fruta in frutas:
    print(fruta.upper())

# Sobre un rango
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 11):      # 1 hasta 10
    print(i)

for i in range(0, 20, 2):   # 0, 2, 4, ..., 18 (paso 2)
    print(i)
```

python

Antipatrón a evitar:

```
# [X] No hacer esto (acceso por índice cuando no se necesita)
for i in range(len(frutas)):
    print(frutas[i])

# [OK] Directo
for fruta in frutas:
    print(fruta)

# [OK] Si necesitas el índice: usar enumerate
for i, fruta in enumerate(frutas):
    print(f"{i}: {fruta}")
```

python

3.3 Bucles -- while

```
intentos = 0
MAX = 3

while intentos < MAX:
    clave = input("Clave: ")
    if clave == "1234":
        print("Correcto")
        break
    intentos += 1
else:
    # El else se ejecuta si el while terminó SIN break
    print("Bloqueado por demasiados intentos")
```

python

3.4 match -- Pattern Matching (Python 3.10+)

```
def describir_http(codigo: int) -> str:
    """Describe un código de respuesta HTTP."""
    match codigo:
        case 200:
            return "OK -- éxito"
        case 400:
            return "Bad Request -- error del cliente"
        case 404:
            return "Not Found -- recurso inexistente"
        case 500:
            return "Internal Server Error"
        case _:
            # wildcard -- cualquier otro caso
            return f"Código HTTP: {codigo}"
```

[match] es especialmente útil con tuplas (destructuring):

```
punto = (1, 0)
match punto:
    case (0, 0):
        print("Origen")
    case (x, 0):
        print(f"En eje X: {x}")
    case (0, y):
        print(f"En eje Y: {y}")
    case (x, y):
        print(f"Punto ({x}, {y})")
```

PARTE 4: Funciones

4.1 Anatomía completa de una función

```
def calcular_imc(peso_kg: float, altura_m: float) -> float:
    """
    Calcula el Índice de Masa Corporal.

    Args:
        peso_kg: Peso en kilogramos (debe ser > 0).
        altura_m: Altura en metros (debe ser > 0).

    Returns:
        IMC como número flotante.

    Raises:
        ValueError: Si algún parámetro es <= 0.
    """
    if peso_kg <= 0 or altura_m <= 0:
        raise ValueError("Peso y altura deben ser positivos")
    return peso_kg / (altura_m ** 2)
```

Elementos obligatorios en este curso:

- [def nombre_funcion(param: tipo) -> tipo_retorno:]
- Docstring con descripción, Args y Returns
- Type hints en todos los parámetros y en el retorno

4.2 Parámetros

```
# Posicionales, keyword y con valor por defecto
def crear_perfil(
    nombre: str,
    edad: int,
    rol: str = "alumno"    # tiene valor por defecto
) -> dict[str, str | int]:
    return {"nombre": nombre, "edad": edad, "rol": rol}

# Llamadas válidas
crear_perfil("Ana", 22)           # usa default
crear_perfil("Bob", 30, "docente") # posicionales
crear_perfil(edad=25, nombre="Carlos") # keyword
```

4.3 Valores de retorno

```
# Retorno simple
def cuadrado(n: int) -> int:
    return n ** 2

# Retorno múltiple (como tupla)
def dividir(a: int, b: int) -> tuple[int, int]:
    """Retorna (cociente, resto)."""
    return a // b, a % b

cociente, resto = dividir(17, 5)    # unpacking

# Retorno opcional (puede ser None)
def buscar(lista: list[int], valor: int) -> int | None:
    for i, x in enumerate(lista):
        if x == valor:
            return i
    return None    # valor no encontrado
```

PARTE 5: Colecciones

5.1 list -- secuencia mutable

```

# Creación
notas = [85, 72, 91, 68, 77]

# Acceso e indexación
notas[0]      # 85 (primer elemento)
notas[-1]     # 77 (último elemento)
notas[1:3]    # [72, 91] (slicing -- NO incluye índice 3)
notas[:3]     # [85, 72, 91] (desde el inicio)
notas[::2]    # [85, 91, 77] (cada dos)
notas[::-1]   # [77, 68, 91, 72, 85] (invertido)

# Modificación
notas.append(95)      # agrega al final
notas.insert(0, 100)  # inserta en posición 0
notas.remove(68)      # elimina por valor
ultimo = notas.pop()  # extrae el último

# Ordenamiento
sorted(notas)          # nueva lista ordenada (original intacta)
notas.sort()           # ordena in-place
notas.sort(reverse=True) # descendente

```

5.2 tuple -- secuencia inmutable

```

# Casos de uso
coordenada = (3.5, -1.2)      # datos que no cambian
color_rgb = (255, 128, 0)     # constante semántica

# Unpacking
x, y = coordenada
r, g, b = color_rgb

# Usar como clave de diccionario (los strings y listas no son hashables)
mapa_pixeles = {(0, 0): "negro", (1, 1): "blanco"}

```

5.3 dict -- mapa clave-valor

```

alumno = {
    "nombre": "Ana",
    "legajo": 12345,
    "notas": [85, 90]
}

# Acceso
alumno["nombre"]          # "Ana" -- KeyError si no existe
alumno.get("email", "sin email") # "sin email" -- seguro, con default

# Modificación
alumno["legajo"] = 99999
alumno["carrera"] = "Sistemas" # agrega nueva clave

# Iteración
for clave, valor in alumno.items():
    print(f"{clave}: {valor}")

for clave in alumno.keys():
    print(clave)

for valor in alumno.values():
    print(valor)

# Verificar existencia
"notas" in alumno # True

```

5.4 set -- conjunto sin duplicados

```

lenguajes = {"Python", "JavaScript", "Python"} # los duplicados se eliminan
# -> {"Python", "JavaScript"}

# Operaciones de conjuntos
a = {"Python", "JS", "C"}
b = {"Python", "Java", "C", "Go"}

a & b    # intersección: {"Python", "C"}
a | b    # unión: {"Python", "JS", "C", "Java", "Go"}
a - b    # diferencia: {"JS"} (en a pero no en b)
a ^ b    # diferencia simetrica: {"JS", "Java", "Go"}

# Eliminar duplicados de una lista
lista_dup = [1, 2, 2, 3, 3, 4]
lista_limpia = list(set(lista_dup)) # [1, 2, 3, 4]

```

5.5 Comprensiones

Las comprensiones son la forma pythónica de crear colecciones a partir de iterables.

```
# Comprensión de lista -- patrón base
cuadrados = [x ** 2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Con filtro
pares = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Dict comprehension
nombres = ["Ana", "Bob", "Carlos"]
longitudes = {n: len(n) for n in nombres}
# {"Ana": 3, "Bob": 3, "Carlos": 6}

# Set comprehension
unicas = {x % 5 for x in range(20)}
# {0, 1, 2, 3, 4}
```

Cuándo usar comprensión vs. for loop:

- Comprensión -> cuando la lógica es simple y cabe en una línea legible
- [for] loop -> cuando hay múltiples pasos o lógica compleja

5.6 Funciones de iteración

> `enumerate()` -- índice automático

```
frutas = ["manzana", "banana", "cereza"]

for i, fruta in enumerate(frutas):
    print(f"{i}: {fruta}")
# 0: manzana
# 1: banana
# 2: cereza

# Empezar desde 1
for i, fruta in enumerate(frutas, start=1):
    print(f"Fruta #{i}: {fruta}")
```

> `zip()` -- iterar en paralelo

```
nombres = ["Ana", "Bob", "Carlos"]
notas = [85, 72, 91]

for nombre, nota in zip(nombres, notas):
    print(f"{nombre}: {nota}")
```

> `sorted()` con `key=`

```

alumnos = [
    {"nombre": "Carlos", "nota": 72},
    {"nombre": "Ana", "nota": 91},
    {"nombre": "Bob", "nota": 85},
]

# Ordenar por nota ascendente (menor a mayor)
por_nota_asc = sorted(alumnos, key=lambda a: a["nota"])
# -> [Carlos(72), Bob(85), Ana(91)]

# Ordenar en forma descendente (mayor a menor)
por_nota_desc = sorted(alumnos, key=lambda a: a["nota"], reverse=True)
# -> [Ana(91), Bob(85), Carlos(72)]

mejor = max(alumnos, key=lambda a: a["nota"])

```

> Builtins de reducción

```

numeros = [3, 1, 4, 1, 5, 9, 2, 6]

len(numeros)      # 8
sum(numeros)      # 31
min(numeros)      # 1
max(numeros)      # 9
sorted(numeros)   # [1, 1, 2, 3, 4, 5, 6, 9]
any(x > 8 for x in numeros) # True (hay al menos uno > 8)
all(x > 0 for x in numeros) # True (todos son positivos)

```

PARTE 6: Módulos, Funciones de Orden Superior y Decoradores

6.1 Módulos y paquetes

```

# Importar un módulo completo
import calculadora
resultado = calculadora.sumar(5, 3)

# Importar una función específica
from calculadora import sumar
resultado = sumar(5, 3)

# Alias útiles
from calculadora import sumar as add

```

Estructura del proyecto (referencia):

```

proyecto/
??? src/
?   ??? __init__.py    <- hace que src/ sea un paquete
?   ??? calculadora.py
??? tests/
?   ??? __init__.py
?   ??? test_calculadora.py
??? requirements.txt

```

Para importar [src/calculadora.py] desde los tests:

```
from src.calculadora import sumar
```

python

6.2 Funciones de orden superior

En Python, las funciones son objetos. Podes:

- Asignarlas a variables
- Pasarlas como argumento
- Devolverlas desde otras funciones

```
# Pasar una función como argumento
def aplicar(func, lista: list[int]) -> list[int]:
    return [func(x) for x in lista]

def duplicar(x: int) -> int:
    return x * 2

resultado = aplicar(duplicar, [1, 2, 3, 4])
# [2, 4, 6, 8]
```

python

> map() -- transformar todos los elementos

```
numeros = [1, 2, 3, 4, 5]
cuadrados = list(map(lambda x: x ** 2, numeros))
# [1, 4, 9, 16, 25]

# Equivalente con comprensión (preferida en Python)
cuadrados = [x ** 2 for x in numeros]
```

python

> filter() -- seleccionar por condición

```
numeros = [1, 2, 3, 4, 5, 6]
pares = list(filter(lambda x: x % 2 == 0, numeros))
# [2, 4, 6]

# Equivalente con comprensión
pares = [x for x in numeros if x % 2 == 0]
```

python

6.3 Lambdas

```
# Una función anónima en una línea
cuadrado = lambda x: x ** 2
cuadrado(5)  # 25

# Uso más común: como argumento en sorted/map/filter
alumnos.sort(key=lambda a: a["nota"])
```

python

Cuándo NO usar lambda:

- Si la función tiene más de una expresión
- Si tenes que darle nombre (usá [def] en ese caso)
- Si queres docstring (las lambdas no la permiten)

6.4 Decoradores básicos

Un decorador es una función que "envuelve" otra función para agregarle comportamiento.

```
import functools

def registrar(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"-> Llamando a {func.__name__}")
        resultado = func(*args, **kwargs)
        print(f"<- {func.__name__} terminó con resultado: {resultado}")
        return resultado
    return wrapper

@registrar
def sumar(a: int, b: int) -> int:
    return a + b

sumar(3, 4)
# -> Llamando a sumar
# <- sumar terminó con resultado: 7
```

python

Decoradores incorporados relevantes:

Decorador	Descripción
<code>[@property]</code>	Convierte un metodo en atributo calculado
<code>[@staticmethod]</code>	Metodo que no recibe <code>[self]</code> ni <code>[cls]</code>
<code>[@functools.wraps]</code>	Preserva el nombre y docstring de la función decorada

PARTE 7: Type Hints (Python 3.10+)

7.1 Que son y por que importan

Los type hints son anotaciones que documentan que tipos espera y devuelve una función. Python no las valida en runtime -- pero Pylance sí las verifica en tiempo de edición, y Ruff puede detectar inconsistencias.

En este curso: son obligatorios en todos los parámetros y retornos de funciones públicas.

7.2 Sintaxis moderna (Python 3.10+)

```
# Tipos primitivos
nombre: str
edad: int
promedio: float
activo: bool

# Colecciones genericas (Python 3.9+ -- sin typing.List)
notas: list[int]          # lista de enteros
mapa: dict[str, float]    # dict con claves str y valores float
par: tuple[int, int]      # tupla de dos enteros
conjunto: set[str]        # conjunto de strings

# Opcional -- Python 3.10+ (sin typing.Optional)
email: str | None = None  # puede ser string o None

# Union de tipos -- Python 3.10+
identificador: int | str  # puede ser int o str

# Any -- cuando el tipo no importa (usar con moderación)
from typing import Any
datos: Any
```

7.3 Anotaciones en funciones

```
def procesar_alumnos(
    nombres: list[str],
    notas: dict[str, int],
    umbral: int = 70
) -> tuple[list[str], list[str]]:
    """Separa alumnos en aprobados y desaprobados.

    Args:
        nombres: Lista de nombres de alumnos.
        notas: Diccionario {nombre: nota}.
        umbral: Nota mínima para aprobar (default: 70).

    Returns:
        Tupla (aprobados, desaprobados) como listas de nombres.
    """
    aprobados = [n for n in nombres if notas.get(n, 0) >= umbral]
    desaprobados = [n for n in nombres if notas.get(n, 0) < umbral]
    return aprobados, desaprobados
```

7.4 Callable -- tipar funciones como argumento

```
from collections.abc import Callable

def aplicar_a_todos(
    lista: list[int],
    func: Callable[[int], int]
) -> list[int]:
    """Aplica func a cada elemento de lista."""
    return [func(x) for x in lista]
```

PARTE 8: PEP 8 + Ruff -- Estilo de Código

8.1 Las reglas más importantes

Regla	Mal	Bien
Indentación	2 espacios o tabs	4 espacios
Nombres de funciones/variables	[MiFunc], [MIFUNC]	[mi_func], [mi_variable]
Nombres de clases	[mi_clase]	[MiClase]
Constantes	[maxIntentos]	[MAX_INTENTOS]
Largo de línea	>88 caracteres	máximo 88 caracteres
Imports	Todo junto, sin orden	estándar -> third-party -> local
Espacios en operadores	[x=1+2]	[x = 1 + 2]

8.2 Ruff -- el linter del curso

Ruff detecta automáticamente violaciones a PEP 8 y muchas otras convenciones. Está instalado en el devcontainer.

```
# Verificar el proyecto
ruff check .

# Corregir automáticamente lo que se puede
ruff check --fix .

# Verificar solo un archivo
ruff check src/calculadora.py
```

bash

Las principales reglas que marca:

Código	Descripción	Ejemplo
[E225]	Falta espacio en operador	[x=1+2]
[F401]	Import no usado	[import os] sin usar [os]
[E501]	Línea muy larga (>88 chars)	--
[N803]	Nombre de parámetro en UpperCase	[def f(Nombre:...)]

[D100]	Falta docstring en módulo	--
[D103]	Falta docstring en función	[def f(): pass]

PARTE 9: Prompting con IA -- Patrón RCTAE

9.1 El patrón obligatorio para PROMPTS.md

Cada vez que uses Copilot o ChatGPT, documentá el prompt completo en [PROMPTS.md] siguiendo el patrón Role + Contexto + Tarea + Restricciones + Ejemplo (RCTAE).

9.2 Ejemplo de prompt bien formado

```
## Prompt 01 -- Función es_primo

**Role:** Eres un tutor de Python 3.13 especialista en algoritmos matemáticos.

**Contexto:** Estoy implementando `src/clasificador.py` para un ejercicio de la materia IF009. Necesito una función que determine si un número es primo.

**Tarea:** Implementá la función `es_primo(n: int) -> bool` con:
- Docstring completa (Google format)
- Type hints
- Lógica eficiente (no verificar hasta n, solo hasta sqrt(n))

**Restricciones:**
- Sin imports. Solo built-ins de Python
- Seguir PEP 8 estrictamente
- Compatible con Python 3.13

**Ejemplo esperado:**
```

```
es_primo(2) # True es_primo(4) # False es_primo(17) # True es_primo(-1) # False
```

```
### Código generado

[pegar código aquí]

### Comprensión línea a línea

- `if n < 2: return False` -> los números negativos, 0 y 1 no son primos por definición
- `for i in range(2, int(n**0.5) + 1)` -> solo necesitamos verificar hasta la raíz cuadrada; si n ...
- `if n % i == 0: return False` -> si i divide exactamente a n, no es primo
- `return True` -> si ninguno de los i lo dividió, es primo
```

9.3 Debugging con IA

Cuando tenes un error, el traceback es el mejor punto de partida para pedirle ayuda a Copilot:

```

**Role:** Eres un tutor de Python experto en debugging.
**Contexto:** Tengo este error en mi desarrollo:
---
Traceback (most recent call last):
  File "src/calculadora.py", line 12
    return a / b
ZeroDivisionError: division by zero
---
La función es: def dividir(a, b): return a / b
**Tarea:** Explicá por que ocurre el error y cómo prevenirlo.
**Restricciones:** Sin try/except. Solo validación con guard clause.

```

Glosario

Termino	Definición
Ágil	Metodología de desarrollo con ciclos iterativos cortos y feedback continuo
CI/CD	Continuous Integration/Deployment -- pipelines automatizados que corren tests en cada push
Comprensión	Sintaxis Python para crear colecciones en una expresión: <code>[[expr for x in iter if cond]]</code>
Decorador	Función que envuelve otra para agregarle comportamiento sin modificarla
DevContainer	Entorno de desarrollo definido como código en <code>[.devcontainer/devcontainer.json]</code>
Docstring	String de documentación al inicio de una función, clase o módulo
f-string	Template de string con interpolación: <code>[f"Hola {nombre}"]</code>

HOF (Higher-Order Function)	Función que recibe o devuelve otras funciones
Immutable	Objeto que no puede modificarse después de crearse (str, tuple, int)
Lambda	Función anónima en una sola expresión: <code>[lambda x: x ** 2]</code>
Linter	Herramienta que detecta errores de estilo y bugs potenciales sin ejecutar el código
PEP	Python Enhancement Proposal -- documentos que definen estándares del lenguaje
PEP 8	Guía de estilo oficial de Python (indentación, nombres, espacios, etc.)
Ruff	Linter/formatter de Python ultrarrápido, reemplaza Flake8 + Black + isort
Type hint	Anotación de tipo en Python: <code>[def f(x: int) -> str:]</code>
Traceback	Reporte de errores de Python que muestra la pila de llamadas
Unpacking	Asignar elementos de una tupla/lista a variables: <code>[x, y = (3, 5)]</code>

Referencias y Recursos

Documentación Oficial

Recurso	URL
Python 3.13 Tutorial	https://docs.python.org/3.13/tutorial/

What's New in Python 3.13	https://docs.python.org/3.13/whatsnew/3.13.html
PEP 8 -- Style Guide	https://peps.python.org/pep-0008/
PEP 484 -- Type Hints	https://peps.python.org/pep-0484/
PEP 636 -- Structural Pattern Matching	https://peps.python.org/pep-0636/
Ruff Documentation	https://docs.astral.sh/ruff/

Bibliografía de la Materia (relevante para este módulo)

- Karymsakova et al. (2025). Practice-Oriented Python Teaching. Open Education Studies -- sobre ciclos T->P cortos y retención
- Fan et al. (2025). AI Pair Programming, Motivation, Anxiety, Performance. IJSTEM -- sobre Copilot en laboratorios
- Alves & Cipriano (2024). Give Me The Code: GPT in First Year CS. arXiv:2411.17855 -- sobre PROMPTS.md como herramienta de comprensión
- Prather (2024). Beyond the Hype: GenAI in CS Education. arXiv:2412.14732 -- sobre IA literacy en programación